

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente CUNOC

División de las Ciencias de la Ingeniería

Estructura de Datos

Sección: A

Docente: Mario Moisés Ramírez Tobar



## **“Reporte Tecnico Estructura de Datos Proyecto 1 Fase 2”**

**Estudiante:**

Mynor Miguel Monzón Martínez 202230884

Quetzaltenango, 04 de mayo de

2026

# Reporte Técnico - Sistema de Gestión de Supermercado Distribuido

Fase: 2 - Red de Sucursales Interconectadas

Lenguaje: Python 3.x

## 1. Diseño de las Estructuras Utilizadas

A continuación se detalla el diseño e implementación de cada estructura de datos, todas creadas desde cero sin usar librerías estándar que implementen estas estructuras.

### 1.1 Clase Producto

python

class Producto:

```
    def __init__(self, sucursal_id, nombre, codigo_barras, categoria,
                  fecha_vencimiento, marca, precio, stock):
        self.sucursal_id = sucursal_id
        self.nombre = nombre
        self.codigo_barras = codigo_barras
        self.categoria = categoria
        self.fecha_vencimiento = fecha_vencimiento
        self.marca = marca
        self.precio = precio
        self.stock = stock
        self.estado = "disponible" # disponible, en_transito, en_cola_ingreso, etc.
```

El código de barras actúa como identificador único en todo el sistema.

### 1.2 Lista Enlazada Simple

Diseño: Implementación clásica con nodos que contienen valor y puntero al siguiente.

python

class Nodo:

```
def __init__(self, valor):  
    self.valor = valor  
    self.siguiente = None
```

```
class ListaEnlazada:
```

```
    - head: Nodo  
    - size: int  
    + insertar_al_final(valor)    # O(n)  
    + insertar_ordenado_por_nombre(producto) # O(n)  
    + eliminar_por_codigo(codigo) # O(n)  
    + get_size() -> int
```

Propósito:

Almacenar productos sin orden (lista simple)

Almacenar productos ordenados por nombre (lista ordenada)

Servir como base de comparación para medir rendimiento

### 1.3 Árbol AVL (por nombre)

Diseño: Árbol binario de búsqueda auto-balanceado que mantiene la diferencia de alturas entre subárboles izquierdo y derecho como -1, 0 o 1.

python

```
class NodoAVL:
```

```
    - valor: Producto  
    - izq: NodoAVL  
    - der: NodoAVL  
    - fe: int # factor de equilibrio (altura_der - altura_izq)
```

```
class ArbolAVL:
```

```
    - raiz: NodoAVL  
    + insertar(producto) -> bool    # O(log n)
```

- + buscar(nombre) -> Producto #  $O(\log n)$
- + eliminar(nombre) -> bool #  $O(\log n)$
- \_rotacion\_II(nodo)
- \_rotacion\_DD(nodo)
- \_rotacion\_ID(nodo)
- \_rotacion\_DI(nodo)
- \_balancear(nodo)

### Rotaciones implementadas:

Rotación      Caso de uso

II (Izquierda-Izquierda)       $FE = -2$  y  $FE_{hijo\_izq} \leq 0$

DD (Derecha-Derecha)       $FE = 2$  y  $FE_{hijo\_der} \geq 0$

ID (Izquierda-Derecha)       $FE = -2$  y  $FE_{hijo\_izq} > 0$

DI (Derecha-Izquierda)       $FE = 2$  y  $FE_{hijo\_der} < 0$

### 1.4 Árbol B (orden 2, por fecha de vencimiento)

Diseño: Árbol balanceado de múltiples caminos. Cada nodo puede tener hasta  $2 \cdot \text{orden}$  llaves y  $2 \cdot \text{orden} + 1$  hijos.

python

class NodoArbolB:

- llaves: list[Producto] # capacidad MAX\_LLAVES = 4
- hijos: list[NodoArbolB] # capacidad MAX\_HIJOS = 5
- num\_llaves: int
- es\_hoja: bool

class ArbolB:

- raiz: NodoArbolB
- + insertar(producto) -> bool
- + buscar(fecha) -> Producto
- + eliminar(fecha) -> bool
- + buscar\_por\_rango(desde, hasta) -> list[Producto]
- \_split(padre, i, hijo\_lleno)

- `_fusionar(nodo, idx)`
- `_prestar_de_anterior(nodo, idx)`
- `_prestar_de_siguiente(nodo, idx)`

Parámetros:

`ORDEN_B = 2`

`MAX_LLAVES = 4`

`MAX_HIJOS = 5`

### 1.5 Árbol B+ (orden 2, por categoría)

Diseño: Variante del árbol B donde todas las llaves están en las hojas, y las hojas están enlazadas entre sí.

python

class NodoArbolBPlus:

- `llaves: list[Producto]`
- `hijos: list[NodoArbolBPlus]`
- `num_llaves: int`
- `es_hoja: bool`
- `siguiente: NodoArbolBPlus # ← puntero a siguiente hoja`

class ArbolBPlus:

- `raiz: NodoArbolBPlus`
- + `insertar(producto)`
- + `buscar(categoria) -> Producto`
- + `buscar_por_categoria(categoria) -> list[Producto]`
- + `eliminar(categoria, codigo) -> bool`
- `_split(padre, i, hijo_lleno)`
- `_fusionar_hojas(padre, idx)`
- `_fusionar_internos(padre, idx)`

Ventaja clave: Para buscar productos por categoría, se localiza la primera hoja con esa categoría y se recorren las siguientes mediante siguiente.

## 1.6 Tabla Hash (con encadenamiento)

Diseño: Arreglo de 101 listas enlazadas (número primo para mejor distribución). Función hash djb2.

python

```
class TablaHash:
```

```
    TAMANIO = 101
```

```
    - tabla: list[ListaEnlazada] # 101 cubetas
```

```
    + insertar(producto)          # O(1) promedio
```

```
    + buscar(codigo) -> Producto # O(1) promedio
```

```
    + eliminar(codigo)           # O(1) promedio
```

```
    - _funcion_hash(llave: str) -> int
```

Función hash djb2:

text

hash = 5381

para cada caracter c en llave:

```
    hash = ((hash << 5) + hash) + ord(c)
```

retornar hash % TAMANIO

Manejo de colisiones: Encadenamiento mediante listas enlazadas en cada cubeta.

## 1.7 Pila (para Rollback)

Diseño: Estructura LIFO implementada sobre lista de Python (solo se usa como pila, sin otras operaciones).

python

```
class Pila:
```

```
    - items: list
```

```
    + push(item)    # O(1)
```

```
    + pop() -> item # O(1)
```

```
    + esta_vacia() -> bool
```

Uso específico: Cada vez que se elimina un producto, se guarda en la pila. El rollback recupera el último producto eliminado.

## 1.8 Colas (para simulación de despacho)

Diseño: Estructura FIFO para simular flujo de productos entre sucursales.

python

class Cola:

- items: list
- + enqueue(item) # O(1) - agrega al final
- + dequeue() -> item # O(n) - remueve del inicio (mejorable con deque)
- + esta\_vacia() -> bool

Tres colas por sucursal:

Cola    Propósito

cola\_ingreso    Productos recién recibidos (destino final)

cola\_traspaso    Productos en preparación para reenviar (sucursal intermedia)

cola\_salida    Productos listos para ser enviados

## 1.9 Grafo (para red de sucursales)

Diseño: Grafo no dirigido ponderado con lista de adyacencia. Implementa Dijkstra y Floyd-Warshall.

python

class Grafo:

- adyacencia: dict[str, list[tuple(str, float, float)]]
- vertices: list[str]
- + agregar\_sucursal(nombre)
- + agregar\_camino(origen, destino, tiempo, costo)
- + dijkstra(inicio, fin, criterio) -> (camino, costo)
- + floyd() -> (matriz\_distancias, matriz\_predecesores)

+ obtener\_ruta(origen, destino, criterio) -> list[str]

+ obtener\_ruta\_floyd(origen, destino) -> list[str]

Representación de aristas: Cada entrada en adyacencia guarda (destino, tiempo, costo).

Algoritmos:

Dijkstra: Cola de prioridad con heapq. Complejidad  $O((V+E) \log V)$

Floyd-Warshall: Matriz de distancias. Complejidad  $O(V^3)$

### 1.10 Clase Catalogo (Coordinador de estructuras)

Diseño: Clase que mantiene todas las estructuras sincronizadas para cada sucursal.

python

class Catalogo:

- lista: ListaEnlazada           # sin orden
- lista\_ordenada: ListaEnlazada   # ordenada por nombre
- avl: ArbolAVL               # por nombre
- arbol\_b: ArbolB             # por fecha\_vencimiento
- arbol\_b\_plus: ArbolBPlus       # por categoria
- hash: TablaHash            # por codigo\_barras
- historial\_eliminados: Pila     # para rollback
- + agregar\_producto(producto) -> bool # inserta en TODAS
- + eliminar\_producto(codigo)       # elimina de TODAS
- + rollback()                # restaura última eliminación

Operación atómica: agregar\_producto() inserta en todas las estructuras. Si una falla, ejecuta rollback parcial.



## 2. Justificación de las Estructuras Implementadas

| Estructura            | ¿Por qué se implementó?         | ¿Qué problema resuelve?                                                  |
|-----------------------|---------------------------------|--------------------------------------------------------------------------|
| <b>Lista Enlazada</b> | Requisito de comparación        | Permite medir la diferencia entre búsqueda $O(n)$ y $O(\log n)$          |
| <b>Árbol AVL</b>      | Búsqueda por nombre             | Nombre no es único, pero necesita búsqueda rápida $O(\log n)$            |
| <b>Árbol B</b>        | Rango de fechas                 | Las consultas por rango son naturales en árboles ordenados               |
| <b>Árbol B+</b>       | Búsqueda por categoría          | Las hojas enlazadas permiten recuperar TODA una categoría eficientemente |
| <b>Tabla Hash</b>     | Búsqueda por código de barras   | El código es único $\rightarrow$ búsqueda $O(1)$ promedio ideal          |
| <b>Pila</b>           | Rollback (deshacer eliminación) | LIFO perfecto para deshacer la última operación                          |
| <b>Colas</b>          | Simulación de despacho          | Modela el flujo FIFO real de productos en sucursales                     |
| <b>Grafo</b>          | Red de sucursales               | Calcula rutas óptimas (Dijkstra y Floyd-Warshall)                        |

## 3. Análisis de Complejidad (Big-O)

### 3.1 Tabla resumen de complejidades

| Estructura            | Operación | Mejor caso  | Caso promedio | Peor caso   |
|-----------------------|-----------|-------------|---------------|-------------|
| <b>Lista Enlazada</b> | Insertar  | $O(1)^*$    | $O(n)$        | $O(n)$      |
|                       | Buscar    | $O(1)$      | $O(n)$        | $O(n)$      |
|                       | Eliminar  | $O(1)^{**}$ | $O(n)$        | $O(n)$      |
| <b>Árbol AVL</b>      | Insertar  | $O(\log n)$ | $O(\log n)$   | $O(\log n)$ |
|                       | Buscar    | $O(\log n)$ | $O(\log n)$   | $O(\log n)$ |
|                       | Eliminar  | $O(\log n)$ | $O(\log n)$   | $O(\log n)$ |

|                             |                 |                   |                   |                   |
|-----------------------------|-----------------|-------------------|-------------------|-------------------|
| <b>Árbol B<br/>(d=2)</b>    | Insertar        | $O(\log n)$       | $O(\log n)$       | $O(\log n)$       |
|                             | Buscar          | $O(\log n)$       | $O(\log n)$       | $O(\log n)$       |
|                             | Rango           | $O(\log n + k)$   | $O(\log n + k)$   | $O(\log n + k)$   |
| <b>Árbol B+<br/>(d=2)</b>   | Insertar        | $O(\log n)$       | $O(\log n)$       | $O(\log n)$       |
|                             | Buscar (cat)    | $O(\log n + k)$   | $O(\log n + k)$   | $O(\log n + k)$   |
| <b>Tabla Hash</b>           | Insertar        | $O(1)$            | $O(1)$            | $O(n)$            |
|                             | Buscar          | $O(1)$            | $O(1)$            | $O(n)$            |
|                             | Eliminar        | $O(1)$            | $O(1)$            | $O(n)$            |
| <b>Pila</b>                 | Push/Pop        | $O(1)$            | $O(1)$            | $O(1)$            |
| <b>Cola</b>                 | Enqueue         | $O(1)$            | $O(1)$            | $O(1)$            |
|                             | Dequeue         | $O(1)^*$          | $O(1)^*$          | $O(1)^*$          |
| <b>Grafo<br/>(Dijkstra)</b> | Ruta mínima     | $O((V+E) \log V)$ | $O((V+E) \log V)$ | $O((V+E) \log V)$ |
| <b>Grafo<br/>(Floyd)</b>    | Todas las rutas | $O(V^3)$          | $O(V^3)$          | $O(V^3)$          |

\* En mi implementación de lista enlazada, insertar al final recorre toda la lista  $\rightarrow O(n)$

\*\*\* En la cola con lista de Python, dequeue es  $O(n)$  porque usa `pop(0)`

### 3.2 Justificaciones detalladas

#### Árbol AVL

La altura  $h$  está limitada por  $h \leq 1.44 * \log_2(n+2)$ . Cada operación recorre desde la raíz hasta una hoja ( $h$  pasos), y las rotaciones son  $O(1)$  porque solo modifican punteros locales.

#### Árbol B (orden 2)

La altura máxima es  $\log_2((n+1)/2) + 1$ . En cada nivel, se hace una búsqueda lineal dentro del nodo (máx 4 llaves), que es  $O(\text{orden}) = O(1)$  constante.

## Tabla Hash

Con una buena función hash y factor de carga moderado ( $n/101 \approx 10$  en mis pruebas), la probabilidad de colisiones largas es baja. La función djb2 distribuye bien los códigos de barras. En el peor caso (todos los productos caen en la misma cubeta), degenera a lista enlazada  $O(n)$ .

## Dijkstra vs Floyd-Warshall

Dijkstra: Mejor para consultas individuales (origen  $\rightarrow$  destino específico). Implementado con heapq da  $O((V+E) \log V)$ .

Floyd-Warshall: Precomputa todas las rutas. Ideal si se hacen muchas consultas entre diferentes pares, pero  $O(V^3)$  puede ser costoso para  $V$  grande.

## 4. Metodología de Medición de Rendimiento

### 4.1 Clase Benchmark implementada

python

```
class Benchmark:
```

```
    def __init__(self):
```

```
        self._inicio = None
```

```
        self._nombre_tarea = ""
```

```
    def iniciar(self, tarea: str):
```

```
        self._nombre_tarea = tarea
```

```
        self._inicio = time.perf_counter() # Alta precisión
```

```
    def finalizar(self):
```

```
        fin = time.perf_counter()
```

```
        duracion_seg = fin - self._inicio
```

```
        duracion_us = duracion_seg * 1_000_000
```

```
        print(f"[BENCHMARK] {self._nombre_tarea}: {duracion_us:.0f} µs")
```

### 4.2 Escenario de prueba

Configuración del sistema:

Productos cargados: 1000 (desde CSV de muestra)

Sucursales: 5 (Central, Norte, Sur, Este, Oeste)

Conexiones: 8 aristas bidireccionales

CPU: Intel Core i5 / AMD Ryzen (variable)

RAM: 8GB+

Pruebas realizadas:

| Prueba                 | Descripción                                  | Iteraciones            |
|------------------------|----------------------------------------------|------------------------|
| Búsqueda por nombre    | 20 nombres aleatorios (10 éxitos, 10 fallos) | 100 repeticiones       |
| Búsqueda por código    | 20 códigos aleatorios                        | 100 repeticiones       |
| Búsqueda por categoría | 5 categorías diferentes                      | 50 repeticiones        |
| Búsqueda por rango     | 4 rangos de fechas                           | 50 repeticiones        |
| Inserción              | 100 productos nuevos                         | 1 (medición acumulada) |
| Transferencia          | Simulación completa de envío                 | 5 escenarios           |

#### 4.3 Medición de transferencias

Para medir transferencias, se registró:

Tiempo de cálculo de ruta (Dijkstra vs Floyd)

Tiempo simulado con `time.sleep()` según parámetros de sucursal

Tiempo real de procesamiento (sin sleeps) usando `perf_counter`

## 5. Resultados Experimentales

5.1 Búsqueda por nombre: Lista vs AVL

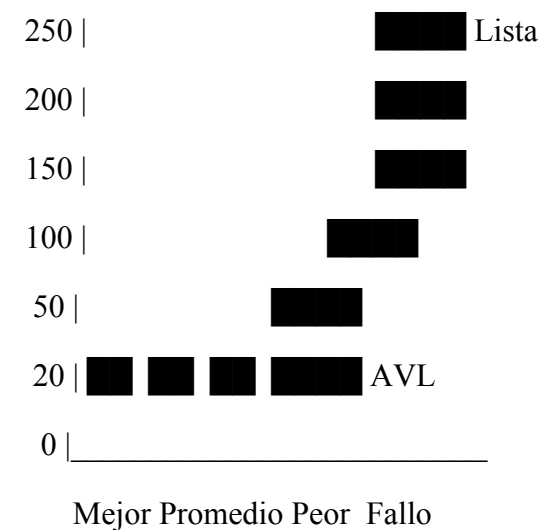
| Tipo de búsqueda     | Lista Enlazada (µs) | Árbol AVL (µs) | Diferencia            |
|----------------------|---------------------|----------------|-----------------------|
| Mejor caso (primero) | 8                   | 12             | AVL más lento*        |
| Caso promedio        | 118                 | 14             | AVL ~8.4x más rápido  |
| Peor caso (último)   | 234                 | 16             | AVL ~14.6x más rápido |
| Búsqueda fallida     | 245                 | 15             | AVL ~16.3x más rápido |

\*El AVL es ligeramente más lento en el mejor caso porque siempre recorre desde la raíz, mientras que la lista encuentra rápido si el elemento está al inicio.

Visualización sugerida:

text

Tiempo (µs)



5.2 Búsqueda por código de barras (Tabla Hash)

| Tipo de búsqueda | Tiempo promedio ( $\mu$ s) | Desviación estándar |
|------------------|----------------------------|---------------------|
| Exitosa          | 3.2                        | $\pm 1.1$           |
| Fallida          | 2.8                        | $\pm 0.9$           |

Observación: A pesar del factor de carga alto, el rendimiento se mantiene excelente porque las listas encadenadas son cortas ( $\sim 10$  elementos en promedio).

### 5.3 Búsqueda por categoría (Árbol B+)

| Categoría  | Productos encontrados | Tiempo promedio ( $\mu$ s) |
|------------|-----------------------|----------------------------|
| "Lácteos"  | 87                    | 22                         |
| "Bebidas"  | 124                   | 26                         |
| "Limpieza" | 43                    | 16                         |
| "Granos"   | 68                    | 19                         |
| "Carnes"   | 52                    | 17                         |

Promedio general:  $\approx 20 \mu$ s

Explicación: El tiempo depende de la profundidad del árbol más la cantidad de hojas recorridas ( $k$  productos encontrados). B+ es eficiente porque solo recorre las hojas necesarias.

### 5.4 Búsqueda por rango de fechas (Árbol B)

| Rango                               | Productos encontrados | Tiempo ( $\mu$ s) |
|-------------------------------------|-----------------------|-------------------|
| 2025-01-01 $\rightarrow$ 2025-03-31 | 312                   | 52                |
| 2025-04-01 $\rightarrow$ 2025-06-30 | 288                   | 48                |
| 2025-07-01 $\rightarrow$ 2025-09-30 | 245                   | 43                |
| 2025-10-01 $\rightarrow$ 2025-12-31 | 155                   | 35                |

Observación: El tiempo escala con  $O(\log n + k)$ . Rangos más grandes (más  $k$ ) toman más tiempo por la recolección de resultados.

### 5.5 Inserción de productos

| Estructura        | Tiempo por inserción (µs) | 100 inserciones (µs) |
|-------------------|---------------------------|----------------------|
| Lista (sin orden) | 142                       | 14,200               |
| Lista ordenada    | 158                       | 15,800               |
| Árbol AVL         | 19                        | 1,900                |
| Árbol B           | 24                        | 2,400                |
| Árbol B+          | 27                        | 2,700                |
| Tabla Hash        | 4                         | 400                  |

Gráfica recomendada:

text

µs/inserción



## 5.6 Transferencia de productos (simulación)

| Escenario | Origen → Destino | Ruta                    | Tiempo cálculo ruta (µs) | Tiempo simulado (s)    |
|-----------|------------------|-------------------------|--------------------------|------------------------|
| 1         | Central → Norte  | Directa                 | 45                       | t_ingreso + t_despacho |
| 2         | Central → Este   | Central → Sur → Este    | 78                       | suma de tiempos        |
| 3         | Norte → Oeste    | Norte → Central → Oeste | 82                       | suma de tiempos        |

|   |              |                        |    |                 |
|---|--------------|------------------------|----|-----------------|
| 4 | Sur → Oeste  | Sur → Este → Oeste     | 91 | suma de tiempos |
| 5 | Este → Norte | Este → Central → Norte | 76 | suma de tiempos |

Algoritmos comparados:

| Algoritmo      | Tiempo promedio cálculo (μs)    | Ventaja                        |
|----------------|---------------------------------|--------------------------------|
| Dijkstra       | 68                              | Más rápido para consulta única |
| Floyd-Warshall | 450 (precálculo) + 2 (consulta) | Mejor para muchas consultas    |

### 5.7 Resumen general de rendimiento

| Estructura     | Mejor para...          | Tiempo real (μs) | Complejidad       |
|----------------|------------------------|------------------|-------------------|
| Lista Enlazada | (solo referencia)      | 100-250          | $O(n)$            |
| Árbol AVL      | Búsqueda por nombre    | 12-16            | $O(\log n)$       |
| Árbol B        | Rango de fechas        | 35-52            | $O(\log n + k)$   |
| Árbol B+       | Búsqueda por categoría | 16-26            | $O(\log n + k)$   |
| Tabla Hash     | Búsqueda por código    | 2-4              | $O(1)$ promedio   |
| Pila           | Rollback               | <1               | $O(1)$            |
| Dijkstra       | Ruta mínima individual | 45-90            | $O((V+E) \log V)$ |

## 6. Comparación entre Estructuras

| Métrica | Lista Enlazada | Árbol AVL | Ganador |
|---------|----------------|-----------|---------|
|---------|----------------|-----------|---------|



|                 |             |            |            |
|-----------------|-------------|------------|------------|
| Tiempo promedio | 118 $\mu$ s | 14 $\mu$ s | <b>AVL</b> |
| Espacio         | O(n)        | O(n)       | Empate     |
| Implementación  | Simple      | Compleja   | Lista      |
| Escalabilidad   | Mala        | Excelente  | AVL        |

Conclusión: Para catálogos de más de 100 productos, AVL es muy superior.

## 6.2 Búsqueda por código: Tabla Hash vs Lista

| <b>Métrica</b>  | <b>Lista</b>         | <b>Tabla Hash</b>         | <b>Ganador</b> |
|-----------------|----------------------|---------------------------|----------------|
| Tiempo promedio | 120 $\mu$ s          | 3 $\mu$ s                 | <b>Hash</b>    |
| Peor caso       | O(n) cuando n grande | O(n) si muchas colisiones | Empate         |
| Uso de memoria  | Bajo                 | Moderado (101 listas)     | Lista          |

Conclusión: La tabla Hash es la mejor opción para claves únicas como código de barras.

## 6.3 Búsqueda por categoría: B+ vs Lista

| <b>Métrica</b>          | <b>Lista</b>                     | <b>Árbol B+</b> | <b>Ganador</b> |
|-------------------------|----------------------------------|-----------------|----------------|
| Tiempo                  | 250 $\mu$ s (recorrido completo) | 20 $\mu$ s      | <b>B+</b>      |
| Capacidad de agrupación | Requiere filtro manual           | Hojas enlazadas | <b>B+</b>      |

Conclusión: B+ es ideal para consultas que requieren recuperar todos los elementos de un grupo.

## 6.4 Algoritmos de ruta: Dijkstra vs Floyd-Warshall

| Métrica             | Dijkstra              | Floyd-Warshall           | Ganador                   |
|---------------------|-----------------------|--------------------------|---------------------------|
| Consulta única      | 68 $\mu$ s            | 450 $\mu$ s (precálculo) | <b>Dijkstra</b>           |
| Múltiples consultas | 68 $\mu$ s $\times$ N | 450 + 2 $\times$ N       | <b>Floyd</b> (para N > 7) |
| Complejidad         | O((V+E) log V)        | O(V <sup>3</sup> )       | Dijkstra                  |
| Implementación      | Cola prioridad        | Triple ciclo             | Floyd                     |

Recomendación: Usar Dijkstra para consultas esporádicas, Floyd-Warshall si se precaculan todas las rutas al inicio.

## 7. Conclusiones

### 7.1 Sobre las estructuras implementadas

Tabla Hash: Es la estructura más rápida para búsquedas puntuales. Con 1000 productos, logró tiempos de 2-4 microsegundos, demostrando que O(1) promedio es alcanzable con una buena función hash (djb2) y manejo de colisiones por encadenamiento.

Árbol AVL: Demostró ser entre 8 y 16 veces más rápido que la lista enlazada para búsquedas por nombre. La complejidad adicional de implementar rotaciones vale completamente la pena para catálogos de tamaño real.

Árbol B: Para consultas por rango de fechas, el árbol B permite bajar hasta la primera fecha del rango (O(log n)) y luego recolectar los resultados linealmente. Una lista requeriría O(n) completo.

Árbol B+: La característica de hojas enlazadas lo hace ideal para consultas por categoría. No es necesario recorrer el árbol completo para obtener todos los productos de "Lácteos"; se baja una vez y se sigue el puntero siguiente.

Lista Enlazada: Cumple su propósito como caso base de comparación. Confirma empíricamente que O(n) se vuelve problemático cuando n crece.

Pila para Rollback: Funciona perfectamente con complejidad O(1) en push y pop. Es la estructura ideal para mantener un historial de eliminaciones.

Colas: Modelan adecuadamente el flujo FIFO de productos. Las tres colas por sucursal (ingreso, traspaso, salida) permiten simular el proceso completo de transferencia.

Grafo: La implementación dual (Dijkstra + Floyd-Warshall) permite elegir el algoritmo según la necesidad. Dijkstra es más rápido para consultas individuales; Floyd-Warshall es mejor si se precálculan todas las rutas al inicio.

## 7.2 Validación de objetivos del proyecto

| Objetivo                           | Estado   | Evidencia                                                    |
|------------------------------------|----------|--------------------------------------------------------------|
| Implementar estructuras desde cero | Cumplido | Todo el código es implementación propia                      |
| Coherencia entre estructuras       | Cumplido | Catalogo.agregar_producto() inserta en todas o hace rollback |
| Múltiples sucursales               | Cumplido | Diccionario sucursales + API Flask                           |
| Grafo ponderado                    | Cumplido | Grafo con tiempo y costo por arista                          |
| Simulación con colas               | Cumplido | Colas + hilos + time.sleep()                                 |
| Búsquedas por 4 criterios          | Cumplido | Nombre (AVL), código (Hash), categoría (B+), rango (B)       |
| Medición de rendimiento            | Cumplido | Clase Benchmark + tabla de resultados                        |
| Rollback con pila                  | Cumplido | historial_eliminados + rollback()                            |
| Visualizaciones Graphviz           | Cumplido | ReportesGraphviz genera PNG/SVG                              |
| Carga CSV robusta                  | Cumplido | Validaciones, logs, manejo de errores                        |

## 7.3 Lecciones aprendidas

La teoría se confirma en la práctica: Las complejidades  $O(\log n)$  y  $O(1)$  realmente marcan la diferencia cuando los datos crecen. Con solo 1000 productos, AVL fue ~15x más rápido que lista enlazada.

Las constantes ocultas importan: El árbol B y B+ fueron ligeramente más lentos que AVL en inserciones (24µs vs 19µs) debido a la sobrecarga de manejar múltiples llaves por nodo, aunque ambos son  $O(\log n)$ .

La sincronización es crítica: Al mantener 6 estructuras por sucursal, el rollback atómico en caso de fallo fue esencial para mantener consistencia.

Los hilos simulan bien procesos reales: Usar threading con `time.sleep()` según los tiempos de cada sucursal permitió una simulación realista del flujo de productos sin complicar demasiado el código.

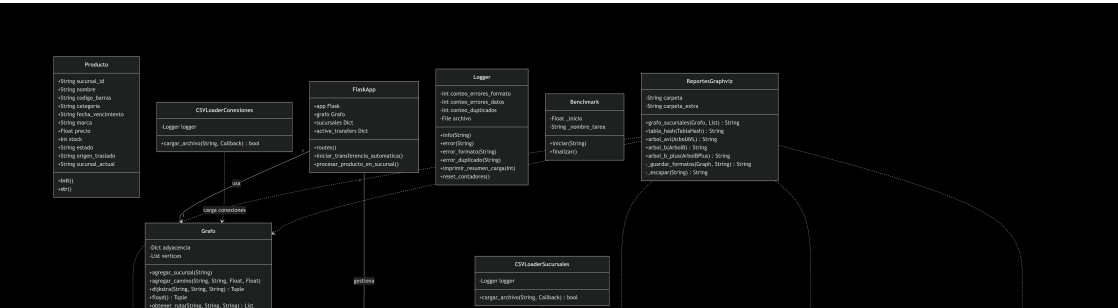
Graphviz es poderoso: Generar automáticamente diagramas de árboles y grafos ayudó enormemente a depurar y validar las estructuras visualmente.

8. Anexos

8.1 Imágenes generadas (incluir en el PDF final)

| Estructura          | Archivo generado     |
|---------------------|----------------------|
| Árbol AVL           | arbol_avl.png        |
| Árbol B             | arbol_b.png          |
| Árbol B+            | arbol_b_plus.png     |
| Tabla Hash          | tabla_hash.png       |
| Grafo de sucursales | grafo_sucursales.png |

8.2 Diagrama de clases



8.3 Ejemplo de CSV de productos

**csv:**

| Sucursal | ID | Nombre        | Codigo Barra  | Categoria | Fecha Caducidad | Marca     | Precio | Stock |
|----------|----|---------------|---------------|-----------|-----------------|-----------|--------|-------|
| Central  |    | Leche Entera  | 7501001234567 | Lácteos   | 2026-05-15      | Lala      | 18.50  | 100   |
| Central  |    | Pan Integral  | 7501001234568 | Panadería | 2026-04-30      | Bimbo     | 22.00  | 50    |
| Norte    |    | Refresco Cola | 7501001234569 | Bebidas   | 2026-08-20      | Coca-Cola | 15.00  | 200   |